

Task Priority Kinematic Control Package: Complete Technical Deep Dive

Package: `task_priority_kinematic_control` Companion GUI package:

`task_priority_kinematic_control_rqt` Scope: source-level explanation of the whole-body task-priority kinematic controller, including function-by-function behavior, URDF loading, kinematics, task computation, solver flow, ROS interfaces, `ros2_control` integration, and RQT tooling.

1. Executive Summary

`task_priority_kinematic_control` implements a ROS 2 whole-body velocity controller for CIRTESUB with two Alpha manipulators. The controller treats the vehicle base and both arms as one generalized velocity vector:

```
qdot = [ active base DOFs | left arm joint velocities | right arm joint velocities ]
```

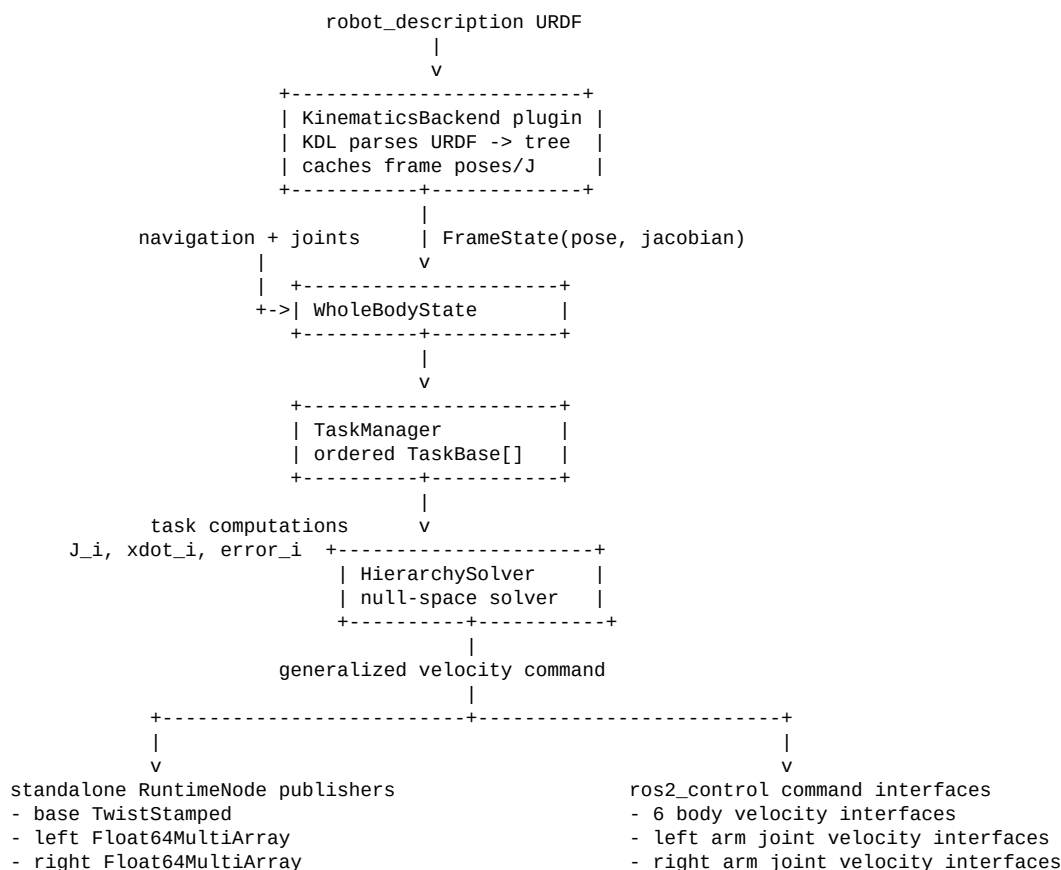
Tasks produce a desired task-space velocity and a Jacobian. The hierarchy solver consumes those tasks in priority order. Higher-priority tasks are solved first; lower-priority tasks are projected into the null space of the already-solved tasks so that they only use remaining freedom.

The package currently provides two execution surfaces:

- `task_priority_runtime`: a standalone ROS 2 node that subscribes to navigation and joint state topics, publishes base and arm velocity topics, and exposes task services.
- `TaskPriorityController`: a `ros2_control` controller plugin that reads hardware state interfaces and writes command interfaces directly.

The real kinematics implementation is `KDLKinematicsBackend`. `PinocchioKinematicsBackend` exists as a plugin placeholder but throws for runtime kinematic queries.

2. Top-Level Architecture



3. Package Contents

Important files and their purpose:

Path	Role
include/.../core/common.hpp	Common math/data structures shared across model, tasks, solver, and backends.
src/core/whole_body_model.cpp	Defines frame names, joint ordering, offsets, and total DOF layout.
src/core/hierarchy_solver.cpp	Implements prioritized null-space velocity solving and velocity saturation.
src/core/task_manager.cpp	Loads task plugins from ROS parameters, sorts them by priority, updates tasks, and handles task runtime changes.
src/core/runtime_node.cpp	Standalone ROS 2 node implementation.
src/core/task_priority_controller.cpp	ros2_control controller plugin implementation.
src/kinematics/kdl_kinematics_backend.cpp	URDF parsing, KDL chain construction, FK, and whole-body Jacobian generation.
src/kinematics/pinocchio_kinematics_backend.cpp	Placeholder backend.
src/tasks/*.cpp	Concrete task plugins.
msg/*.msg	Status, diagnostics, output, and task state messages.
srv/*.srv	Task management and backend management services.
config/*.yaml	Example runtime-node and ros2_control parameter sets.
task_priority_kinematic_control_rqt/.../task_priority_panel.py	RQT panel for listing, enabling, reordering, and sending goals to tasks.

4. Core Data Model

4.1 SolverMethod

Defined in common.hpp.

Values:

- kDls: damped least squares. This is the default and most robust near singularities.
- kPinv: pseudo-inverse based on Eigen's complete orthogonal decomposition.
- kSvd: SVD pseudo-inverse over a weighted Jacobian.

4.2 FrameState

Represents the current kinematic state of a frame.

Fields:

- pose: world-frame Eigen::Isometry3d pose of the frame.
- jacobian: $6 \times \text{total_dofs}$ matrix. Top 3 rows are translational velocity contribution; bottom 3 rows are angular velocity contribution.

4.3 WholeBodyState

Runtime state consumed by kinematics and tasks.

Fields:

- base_position: world position from the navigator message.
- base_rpy: roll, pitch, yaw from the navigator message.
- joint_positions: ordered according to WholeBodyModel::all_joint_names().
- joint_velocities: same order as positions.
- navigation_valid: true after navigation data has arrived.
- joints_valid: true after joint data has arrived.

4.4 WholeBodyCommand

Solver output.

Fields:

- `generalized_velocity`: full command in solver ordering.
- `base_velocity`: standalone runtime convenience slice for the active base DOFs.
- `left_arm_velocity`: standalone runtime convenience slice for left arm joints.
- `right_arm_velocity`: standalone runtime convenience slice for right arm joints.

4.5 TaskComputation

The object returned by every task update.

Fields:

- `jacobian`: task Jacobian.
- `desired_velocity`: task-space command.
- `error`: diagnostic or control error vector.
- `frame_pose`: current pose when the task is tied to a frame.
- `active`: whether the solver should consume the task.
- `has_frame_pose`: whether `frame_pose` is valid for publication.
- `frame_id`: frame controlled by the task.
- `status_message`: human-readable state such as `tracking`, `disabled`, or `waiting_for_joints`.

4.6 skew(v)

Builds the skew-symmetric matrix used for cross-product linearization:

```
skew(v) * w = v x w
```

The KDL backend uses it to build the base angular contribution to the translational velocity of a frame:

```
v_frame = v_base - skew(p_frame - p_base) * omega_base
```

5. Whole-Body Model

Class: `WholeBodyModel` Implementation: `src/core/whole_body_model.cpp`

The model is not a dynamics model. It is a compact description of frames, joint ordering, and generalized velocity layout.

configure(...)

Stores:

- world frame
- base frame
- left and right tip frames
- left arm joint list
- right arm joint list
- active base DOF indices

It also concatenates left and right joint names into `all_joint_names_`. This concatenated order is the canonical order for all joint vectors in the package.

total_dofs()

Returns:

```
active_base_dofs.size + left_arm_joints.size + right_arm_joints.size
```

base_dofs()

Returns the number of active base DOFs. The base can expose all 6 DOFs or a subset such as [0, 1, 2, 5].

Base DOF index convention:

Index	Meaning
0	linear x
1	linear y
2	linear z
3	angular x / roll rate
4	angular y / pitch rate
5	angular z / yaw rate

left_arm_dofs() and right_arm_dofs()

Return the number of configured joints for each arm.

Frame and joint getters

world_frame(), base_frame(), left_tip_frame(), right_tip_frame(), left_arm_joints(), right_arm_joints(), and active_base_dofs() return the configured identifiers and DOF list.

left_offset() and right_offset()

Return the starting column in the generalized velocity vector:

```
left_offset = base_dofs
right_offset = base_dofs + left_arm_dofs
```

joint_index(joint_name)

Searches the concatenated joint list. It returns the zero-based index inside the joint-only vector, or -1 if the joint is unknown.

The solver column for a joint is:

```
solver_column = base_dofs + joint_index
```

all_joint_names()

Returns the left joints followed by right joints. Runtime node joint-state extraction, ros2_control state interface ordering, and joint-space tasks all depend on this order.

6. URDF Reading And Kinematics Flow

There are two equivalent URDF entry points:

- Standalone node: `RuntimeNode::resolve_robot_description()`
- `ros2_control` plugin: `TaskPriorityController::resolve_robot_description()`

Both follow the same logic:

```
if local parameter robot_description is non-empty:
    use it directly
else:
    connect to robot_description_source_node parameter service
    wait robot_description_wait_timeout_sec
    read parameter "robot_description"
    reject empty or non-string values
```

Once the string is resolved:

```
rebuild_backend()
backend_plugin_name = parameter backend_plugin
create pluginlib instance
backend.configure(model, robot_description, logger)
```

URDF And KDL Backend Diagram

```
robot_description string
  |
  v
kdl_parser::treeFromString(...)
  |
  v
KDL::Tree
  |
  +--> for each segment in tree:
  |     try tree.getChain(base_frame, segment_frame)
  |     save ChainData(frame_id, KDL::Chain, joint_names)
  |
  v
per update(state):
  for each ChainData:
    q = q_for_chain(chain, state)
    forward integrate KDL segment poses
    collect joint origins, axes, joint types
    world_to_base = base pose from navigator
    frame pose = world_to_base * base_to_tip
    frame jacobian = [base contribution | arm joint contribution]
    cache FrameState
```

7. Kinematics Backend Interface

Class: KinematicsBackend Header: include/.../kinematics/kinematics_backend.hpp

`configure(model, robot_description, logger)`

Every backend receives:

- a `WholeBodyModel` containing frame names and joint ordering
- a full URDF string
- a ROS logger

`update(state)`

Refreshes backend-specific caches for the current base and joint state.

`get_frame_state(frame_id)`

Returns the current `FrameState` for a frame. Tasks call this to get poses and Jacobians.

`get_relative_transform(from_frame, to_frame)`

Returns the transform from `from_frame` to `to_frame`.

`name()`

Returns a short backend name, for example `kdl` or `pinocchio`.

8. KDL Backend Function-by-Function

File: `src/kinematics/kdl_kinematics_backend.cpp`

Helper: `describe_chain(chain)`

Builds a readable string of KDL segment names and joint names. It is used only for startup logging of left and right chains.

Helper: `base_pose_from_state(state)`

Creates an `Eigen::Isometry3d` from navigator position and RPY.

Rotation order:

$R = R_z(\text{yaw}) * R_y(\text{pitch}) * R_x(\text{roll})$

Helper: to_eigen(KDL::Vector)

Converts a KDL::Vector to Eigen::Vector3d.

Helper: is_revolute(joint_type)

Returns true for KDL rotational joint types: RotAxis, RotX, RotY, and RotZ.

Helper: is_prismatic(joint_type)

Returns true for KDL translational joint types: TransAxis, TransX, TransY, and TransZ.

KDLKinematicsBackend::configure(model, robot_description, logger)

Stores the model and logger, then parses the URDF:

```
kdl_parser::treeFromString(robot_description, tree_)
```

If parsing fails, it throws.

It then iterates all KDL tree segments and tries to build a chain from the configured base frame to each segment frame. Frames that cannot be reached from the base frame are skipped. The resulting chains_ map lets any task request any reachable frame, not only the two end effectors.

Finally it logs the configured left and right chains. Because it uses chains_.at(left_tip_frame) and chains_.at(right_tip_frame), configuration will throw if either configured tip frame is not reachable in the URDF.

KDLKinematicsBackend::update(state)

Copies the latest state to last_state_, clears cached_frames_, and recomputes a FrameState for every chain.

This is where all frame poses and Jacobians are refreshed for the current control tick.

KDLKinematicsBackend::get_frame_state(frame_id)

Looks up frame_id in cached_frames_. If it does not exist, it throws a runtime error. Tasks therefore require the backend to have been updated and the frame to be reachable from the base frame.

KDLKinematicsBackend::get_relative_transform(from_frame, to_frame)

Reads both frame poses from the cache and returns:

```
T_from_to = inverse(T_world_from) * T_world_to
```

KDLKinematicsBackend::name()

Returns kdl.

KDLKinematicsBackend::to_isometry(frame)

Converts a KDL frame into an Eigen isometry by copying the 3x3 rotation and 3D translation.

KDLKinematicsBackend::build_chain_data(tree, base_frame, tip_frame)

Calls:

```
tree.getChain(base_frame, tip_frame, data.chain)
```

Then extracts every non-fixed joint name from the chain into data.joint_names.

KDLKinematicsBackend::q_for_chain(chain, state)

Builds the joint vector for one chain. For each joint in chain.joint_names, it asks the model for the global joint index. If the joint exists and the state vector is long enough, it copies the corresponding joint position. Unknown joints remain at zero.

KDLKinematicsBackend::compute_frame_state(frame_id, chain, state)

This is the main kinematic calculation.

Step-by-step:

1. Build q for this chain.
2. Start with identity `base_to_tip_kdl`.
3. Iterate KDL segments.
4. Before applying each movable joint transform, record the joint origin and axis in the base frame.
5. Apply the segment pose at the current joint value.
6. Convert the final `base_to_tip_kdl` to Eigen.
7. Build `world_to_base` from navigation state.

8. Compute world pose:

```
T_world_tip = T_world_base * T_base_tip
```

9. Allocate a zero Jacobian of size $6 \times \text{total_dofs}$.

10. Build base Jacobian:

```
p = p_tip_world - p_base_world
J_base = [ I   -skew(p) ]
          [ 0     I     ]
```

11. Copy only the active base DOF columns into the first solver columns.
12. For each chain joint that belongs to the whole-body model, compute its world axis and world joint origin.
13. For revolute joints:

```
J_linear = axis_world x (p_tip_world - p_joint_world)
J_angular = axis_world
```

14. For prismatic joints:

```
J_linear = axis_world
J_angular = 0
```

15. Place each joint column at `base_dofs + model.joint_index(joint_name)`.

The result is a whole-body Jacobian with base columns and arm columns aligned with the solver vector.

9. Pinocchio Backend

File: `src/kinematics/pinocchio_kinematics_backend.cpp`

This backend is currently a placeholder.

`configure(...)`

Stores model/logger and warns that Pinocchio is not implemented.

`update(...)`

No-op.

`get_frame_state(...)` and `get_relative_transform(...)`

Both throw runtime errors.

`name()`

Returns `pinocchio`.

Practical consequence: use `task_priority_kinematic_control/KDLKinematicsBackend` for runtime execution.

10. Task Plugin Model

All tasks implement TaskBase.

The common task lifecycle is:

```
TaskManager loads plugin
      |
      v
task.configure(task_id, plugin_name, task params, TaskContext{model})
      |
      v
each control tick:
    task.update(WholeBodyState, KinematicsBackend) -> TaskComputation
```

The solver does not know task semantics. It only consumes TaskComputation.

11. Task Base Function-by-Function

File: src/tasks/task_base.cpp

TaskBase::set_pose_goal(...)

Default implementation rejects pose goals by returning false.

TaskBase::set_joint_target(...)

Default implementation rejects joint targets and writes Task does not accept joint targets.

TaskBase::set_enabled(...)

Default implementation returns false. Concrete common tasks override it.

TaskBase::reset()

Default no-op.

TaskBase::current_target()

Default returns an empty vector.

TaskBaseCommon::configure_common(...)

Stores common metadata:

- task id
- plugin name
- group
- priority
- enabled flag
- task context with shared model

Priority is read as a double parameter then cast to uint32_t; lower values run earlier.

TaskBaseCommon::set_pose_goal(goal)

Stores a PoseStamped runtime goal and marks has_pose_goal_ = true.

TaskBaseCommon::set_joint_target(...)

Rejects joint targets by default.

TaskBaseCommon::set_enabled(enabled)

Changes the task enabled flag.

TaskBaseCommon::reset()

Currently no-op. It is a hook for future stateful tasks.

TaskBaseCommon::build_status()

Builds a TaskStatus message with id, plugin, group, priority, enabled, active, status text, and target type. Base implementation reports target type none.

Metadata getters

id(), plugin_name(), group(), priority(), set_priority(), and enabled() provide task metadata to managers, services, and publishers.

Parameter helpers

get_string_param, get_double_param, get_bool_param, get_double_array_param, and get_string_array_param read from the per-task parameter map and return defaults when the key is absent.

12. Concrete Tasks

12.1 EndEffectorPositionTask

File: src/tasks/end_effector_position_task.cpp

Purpose: track the Cartesian position of a frame, normally an end effector.

configure(...)

Reads:

- frame_id, defaulting to left tip frame
- gain, default [1, 1, 1]
- default_goal, default [0, 0, 0]

update(state, backend)

If disabled, returns inactive with disabled.

Otherwise:

1. Gets FrameState from backend.
2. Chooses target position from runtime pose goal if available, otherwise default_goal_.
3. Computes:

```
error = target_position - current_position
desired_velocity = diag(gain) * error
jacobian = frame.jacobian.topRows(3)
```

4. Marks frame pose available and status tracking.

12.2 EndEffectorOrientationTask

File: src/tasks/end_effector_orientation_task.cpp

Purpose: track orientation of a frame.

Helper: quaternion_error(target, current)

Computes:

```
q_err = target * conjugate(current)
error = axis(q_err) * angle(q_err)
```

configure(...)

Reads:

- frame_id, defaulting to left tip frame

- gain, default [1, 1, 1]

The default target quaternion is identity.

update(state, backend)

If disabled, returns inactive.

Otherwise:

1. Gets frame pose/Jacobian.
2. Chooses target quaternion from runtime pose goal or identity.
3. Computes orientation error via `quaternion_error`.
4. Computes angular desired velocity:

```
desired_velocity = diag(gain) * orientation_error
jacobian = frame.jacobian.bottomRows(3)
```

12.3 EndEffectorPoseTask

File: `src/tasks/end_effector_pose_task.cpp`

Purpose: track full 6D pose of an end effector.

Helper: `quaternion_error(...)`

Same axis-angle orientation error used by orientation task.

configure(...)

Reads:

- frame_id, defaulting to left tip frame
- gain, default [1, 1, 1, 1, 1, 1]

update(state, backend)

If disabled, returns inactive.

Otherwise:

1. Gets `FrameState`.
2. If a runtime pose goal exists, position error is target minus current.
3. If no runtime pose goal exists, position target is current pose, so position error is zero.
4. If a runtime pose goal exists, orientation error is target-current quaternion error.
5. If no runtime pose goal exists, orientation error remains zero.
6. Computes:

```
desired_velocity = diag(gain) * [position_error; orientation_error]
jacobian = full 6 x total_dofs frame jacobian
```

Important behavior: as implemented, `EndEffectorPoseTask` does not use a configured default pose target; it only moves when a runtime pose goal has been provided.

12.4 FramePoseTask

File: `src/tasks/frame_pose_task.cpp`

Purpose: track full pose of any frame, including the base frame.

Helper: `quaternion_error(...)`

Same axis-angle error.

Helper: `base_pose_from_state(state)`

Builds an Eigen pose for the base from navigator position/RPY.

Helper: `base_jacobian_for_model(model)`

Builds a $6 \times \text{total_dofs}$ Jacobian for the base frame. It maps active base DOFs directly to their corresponding task-space coordinate.

`configure(...)`

Reads:

- `frame_id`, defaulting to base frame
- `gain`, default `[1, 1, 1, 1, 1, 1]`

`update(state, backend)`

If disabled, returns inactive.

If `frame_id` equals the configured base frame, it uses navigation state and the direct base Jacobian. Otherwise it asks the backend for frame state.

It then behaves like `EndEffectorPoseTask`: runtime pose goal drives position and orientation error; without a runtime pose goal, error is zero.

12.5 JointNominalTask

File: `src/tasks/joint_nominal_task.cpp`

Purpose: bias the solution toward a nominal joint configuration.

`configure(...)`

Reads:

- `joint_names`, default all model joints
- `target`, default zeros with length `joint_names`
- `gain`, default 0.3 for every configured joint

`update(state, backend)`

Active only if enabled and joint positions exist.

For every configured joint:

```
solver_column = base_dofs + model.joint_index(joint)
J(row, solver_column) = 1
error(row) = target(row) - current_joint_position
desired_velocity(row) = gain(row) * error(row)
```

This task only commands arm joints, not base DOFs.

`set_joint_target(target, message)`

Accepts a runtime joint target only if the target vector length exactly matches `joint_names_`.

`build_status()`

Extends the common status with:

```
target_type = "joint_array"
joint_names = configured joint_names
```

`current_target()`

Returns the current nominal joint target as a double vector.

12.6 JointLimitsTask

File: `src/tasks/joint_limits_task.cpp`

Purpose: repel joints away from their lower and upper limits when they enter a safety margin.

`configure(...)`

Reads:

- `margin`, default 0.1
- `gain_scalar`, default 0.5
- `lower_limits`, default zeros
- `upper_limits`, default zeros

`update(state, backend)`

Active only if enabled and joint positions exist.

For each joint:

```
J(i, base_dofs + i) = 1

if upper > lower:
    if q < lower + margin:
        cmd = gain * ((lower + margin) - q)
    else if q > upper - margin:
        cmd = -gain * (q - (upper - margin))
    else:
        cmd = 0

error(i) = cmd
desired_velocity(i) = cmd
```

This task does not compute a classical position error. Its error vector stores the repulsive velocity command for diagnostics.

12.7 BaseYawTask

File: `src/tasks/base_yaw_task.cpp`

Purpose: control base yaw toward a configured scalar target.

Helper: `wrap_angle(angle)`

Wraps an angle into $[-\pi, \pi]$.

Helper: `base_pose_from_state(state)`

Builds the base pose for task-state publication.

`configure(...)`

Reads:

- `target_yaw`, default 0
- `gain_scalar`, default 1

`update(state, backend)`

Active only if enabled, navigation is valid, and the model has at least one active base DOF.

It creates a one-row Jacobian. The column is nonzero only for active base DOF 5:

```
J(0, yaw_column) = 1
error = wrap_angle(target_yaw - current_yaw)
desired_velocity = gain * error
```

If yaw is not present in `active_base_dofs`, this task remains active but its Jacobian row will be all zeros, so the solver cannot realize it.

13. Task Manager

File: `src/core/task_manager.cpp`

Constructor

Stores the ROS parameter interface and logger, then creates a pluginlib loader for `task_priority_kinematic_control::TaskBase`.

`configure(context)`

Reads the `task_ids` parameter. For each task id:

1. Gets all parameters under `tasks.<id>`.
2. Requires a `plugin` parameter.
3. Creates the plugin instance.
4. Calls `task->configure(...)`.
5. Adds it to `tasks_`.

After loading, tasks are sorted by ascending priority.

`update_all(state, backend)`

Calls `update` on every task in current priority order and returns all `TaskComputation` objects.

`set_task_enabled(task_id, enabled, message)`

Finds the task and changes its enabled flag. Returns false if the task id is unknown.

`set_task_pose_goal(task_id, goal, message)`

Forwards a runtime `PoseStamped` goal to the matching task.

`set_task_joint_target(task_id, target, message)`

Forwards a runtime joint target vector to the matching task.

`reorder_tasks(ordered_ids, message)`

Requires the supplied list to contain exactly the same number of tasks as the current manager. It assigns new priorities according to list order and replaces `tasks_` with that order.

`get_task_statuses()`

Builds a status message for every task. It includes a compatibility rule: if a task reports target type none and its plugin name contains `PoseTask`, the status target type is set to pose.

`tasks()`

Returns the internal shared pointers. Used by runtime node/controller setup to create per-task subscribers and publishers.

`reset()`

Calls `reset()` on every task.

`parameters_for_task(task_id)`

Uses `list_parameters` under `tasks.<task_id>` and converts full parameter names into short names like `plugin`, `enabled`, `gain`, or `frame_id`.

14. Hierarchy Solver

File: `src/core/hierarchy_solver.cpp`

The solver is velocity-level and priority-ordered.

Solver Diagram

ordered tasks: $T_0, T_1, T_2, \dots$

```
qdot = 0
N = I
```

```
for each task  $T_i$ :
    if inactive or wrong Jacobian width:
        skip
```

```
JN =  $J_i * N$ 
inv = weighted_inverse(JN)
residual =  $\dot{x}_i - J_i * \dot{q}$ 
 $\dot{q} = \dot{q} + inv * residual$ 
N = N - inv * JN
```

```
 $\dot{q} = \text{clamp}(\dot{q}, -\text{velocity\_limits}, +\text{velocity\_limits})$ 
```

configure(dofs)

Sets the solver DOF count, initializes all weights to 1, and initializes all velocity limits to 1.

set_method(method)

Selects DLS, pseudo-inverse, or SVD.

set_damping(damping)

Stores DLS damping with a minimum of $1e-6$.

set_weights(weights)

Stores the DOF weight vector. Higher weights make a DOF more expensive in the weighted inverse.

set_velocity_limits(limits)

Stores per-DOF absolute velocity limits.

last_condition_estimate()

Returns the last condition estimate computed inside `weighted_inverse`.

solve(tasks)

Creates a zero generalized velocity and identity null-space projector. Each active task is projected through the current projector, solved, and accumulated into the output. Finally the generalized velocity is clamped.

Important details:

- A task is skipped if `active == false`.
- A task is skipped if its Jacobian column count does not equal solver DOFs.
- The residual uses the original task Jacobian:

```
desired_velocity - task.jacobian * current_generalized_velocity
```

This means lower-priority tasks only try to correct what remains after higher-priority motion has already been assigned.

weighted_inverse(jacobian)

Builds:

```
W_inv = diag(1 / max(weights, 1e-6))
metric = J * W_inv * J^T
```

It estimates condition number from singular values of `metric`.

Modes:

- `kPinv`: $W_inv * J^T * \text{pseudoInverse}(\text{metric})$
- `kSvd`: SVD inverse of $J * \text{sqrt}(W_inv)$ mapped back by $\text{sqrt}(W_inv)$

- $\mathbf{kDls} = \mathbf{W_inv} * \mathbf{J}^T * \text{inverse}(\text{metric} + \lambda * \mathbf{I})$

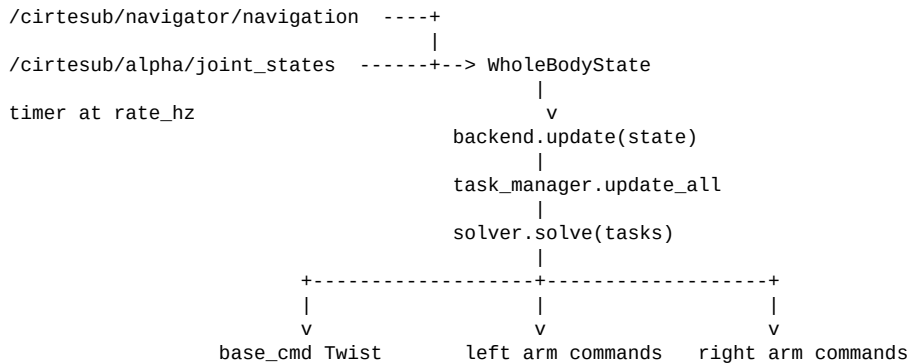
apply_velocity_limits(command)

Clamps every command component to $[-\text{limit}, \text{limit}]$, with a minimum internal limit of $1e-9$.

15. Standalone Runtime Node

Class: RuntimeNode File: src/core/runtime_node.cpp Executable: task_priority_runtime

Runtime Node Data Flow



Constructor

Creates the model and task manager, declares parameters, configures model/backend/solver/tasks, creates publishers/subscribers/services/timer, and installs a dynamic parameter callback.

Helper: declare_if_missing(node, name, default_value)

Declares a parameter only when it was not already declared through node options or launch overrides.

Helper: parse_solver_method(name)

Maps pinv to kPinv, svd to kSvd, and all other strings to kDls.

Helper: solver_method_name(method)

Inverse mapping for status publication.

Helper: parse_active_base_dofs(values, logger)

Validates that every active base DOF index is between 0 and 5 and returns the integer vector.

declare_parameters()

Declares all runtime-node parameters with defaults, including backend, URDF source, frames, joints, solver settings, topics, rate, and task ids.

configure_model()

Reads frame/joint/base-DOF parameters and calls `WholeBodyModel::configure`.

configure_backend()

Calls `rebuild_backend()`.

resolve_robot_description()

Loads URDF locally or from a remote node parameter service, as described in Section 6.

rebuild_backend()

Creates the backend plugin named by `backend_plugin` and configures it with model and URDF.

configure_solver()

Initializes the solver with total DOFs, method, DLS lambda, optional DOF weights, and optional velocity limits.

configure_publishers()

Creates:

- base command publisher
- left arm command publisher
- right arm command publisher
- hierarchy_state
- solver_diagnostics

configure_subscribers()

Creates navigation and joint state subscribers. It also creates per-task target subscribers:

- pose tasks: task_priority/tasks/<task_id>/target
- joint nominal tasks: task_priority/tasks/<task_id>/joint_target

configure_services()

Creates:

- set_task_enabled
- reorder_tasks
- list_tasks
- switch_backend
- reset_solver

configure_timer()

Creates the periodic control timer. Minimum internal rate is 1 Hz.

navigator_callback(msg)

Updates base position/RPY and marks navigation valid.

joint_state_callback(msg)

Builds ordered joint position/velocity vectors by matching incoming joint names against model->all_joint_names().

timer_callback()

If navigation or joints are not valid, publishes status and returns.

Otherwise:

1. backend_->update(state_)
2. task_manager_->update_all(state_, *backend_)
3. solver_.solve(task_outputs)
4. Slice generalized velocity into base, left arm, and right arm pieces.
5. Publish base TwistStamped.
6. Publish left and right Float64MultiArray.
7. Publish status.

`publish_status()`

Publishes hierarchy state and solver diagnostics. `ready` is true only when navigation and joint data are both valid.

`on_parameters_set(parameters)`

Dynamically accepts updates to:

- `dls_lambda`, requiring positive value
- `solver_method`
- `dof_weights`, requiring all positive
- `velocity_limits`, requiring all positive

16. `ros2_control` Controller

Class: `TaskPriorityController` File: `src/core/task_priority_controller.cpp`

This plugin provides the same control logic but integrates into the `ros2_control` update loop.

`ros2_control` Update Diagram

```
controller_manager update()
  |
  v
TaskPriorityController::update(time, period)
  |
  +--> read latest Navigator from realtime buffer
  +--> read joint state_interfaces
  +--> backend.update(state)
  +--> task_manager.update_all(state, backend)
  +--> solver.solve(tasks)
  +--> write command_interfaces
  +--> publish hierarchy/task/output diagnostics
```

Helper: `base_dof_name(dof)`

Maps base DOF index to diagnostic names such as `base.linear.x` and `base.angular.z`.

Helper: `auto_declare_if_missing(controller, name, default_value)`

Declares controller parameters during `on_init` if they are missing.

Helper: `parse_solver_method(name)` and `solver_method_name(method)`

Same mapping as runtime node.

Helper: `pose_msg_from_isometry(pose)`

Converts an Eigen pose into a `geometry_msgs/Pose`.

Helper: `vector_from_eigen(vector)`

Copies an Eigen vector into `std::vector<double>` for ROS messages.

Helper: `parse_active_base_dofs(values, logger)`

Same active-base validation as runtime node.

Helper: `append_joint_velocity_interfaces(controller, parameter_name, interface_names)`

Reads a joint-name parameter and appends `<joint>/velocity` command interface names.

Helper: `append_joint_state_interfaces(controller, parameter_name, interface_type, interface_names)`

Reads joint names and appends <joint>/<interface_type> state interface names.

`on_init()`

Declares all controller parameters, then calls `declare_task_parameters()`.

`declare_task_parameters()`

For every configured task id, declares common task parameter names such as plugin, enabled, priority, group, frame, gains, limits, targets, and yaw.

`command_interface_configuration()`

Requests individual command interfaces:

1. six body velocity interfaces under `body_velocity_controller_name`
2. left arm <joint>/velocity
3. right arm <joint>/velocity

Even if `active_base_dofs` uses fewer than six base DOFs, the `ros2_control` controller still writes six body velocity interfaces and fills inactive DOFs with zero.

`state_interface_configuration()`

Requests individual position and velocity state interfaces for all left and right arm joints.

`on_configure(...)`

Creates model, task manager, backend plugin loader, backend, solver, tasks, navigator subscriber, and external ROS services/publishers/subscribers.

It also logs the generalized velocity ordering. That ordering is critical for interpreting solver output.

`configure_model()`

Calls `WholeBodyModel::configure` with already-read frame/joint/base-DOF parameters.

`resolve_robot_description()`

Loads URDF locally or from a remote parameter service.

`rebuild_backend()`

Creates and configures the kinematics backend plugin.

`configure_solver()`

Configures DOF count, solver method, damping, weights, and velocity limits.

`refresh_task_manager()`

Rebuilds the task manager and loads tasks from parameters.

`configure_external_interfaces()`

Creates:

- `/hierarchy_state`
- `task_priority/output`
- `/list_tasks`
- `/set_task_enabled`

- /set_task_disabled
- /reorder_tasks
- per-task state publishers
- per-task pose or joint target subscribers

publish_hierarchy_state(time)

Publishes backend, solver method, readiness, and task statuses.

publish_task_states(task_outputs)

For each task, publishes TaskState with enabled/active flags, frame pose, error vector, desired velocity vector, and current target.

publish_controller_output(time, command)

Publishes a structured view of the solver command:

- generalized velocity names and values
- six base velocity names and values
- left joint names and velocities
- right joint names and velocities

on_activate(...)

Calls reset_commands() and succeeds.

on_deactivate(...)

Calls reset_commands() and succeeds.

reset_commands()

Writes zero to every command interface.

update(time, period)

The real-time control tick:

1. Verifies command interface count equals $6 + \text{left_joints} + \text{right_joints}$.
2. Reads the latest navigator message from a realtime buffer. If missing, zeroes commands and returns OK.
3. Reads and validates all joint state interfaces.
4. Builds wholeBodyState.
5. Locks task mutex.
6. Updates backend.
7. Updates all tasks.
8. Solves hierarchy.
9. Publishes hierarchy and task states.
10. Builds a six-DOF base command vector with inactive base DOFs set to zero.
11. Writes six base command interfaces.
12. Writes left arm command interfaces.
13. Writes right arm command interfaces.
14. Publishes ControllerOutput.

17. Runtime Node vs ros2_control Controller

Aspect	RuntimeNode	TaskPriorityController
Input navigation	topic subscriber	topic subscriber into realtime buffer
Input joints	sensor_msgs/JointState	ros2_control state interfaces
Output base	TwistStamped topic	six command interfaces
Output arms	two Float64MultiArray topics	joint velocity command interfaces
Timing	ROS wall timer	controller manager update loop
Task services	relative service names	absolute service names
Main use	quick integration / topic-based control	hardware/simulation through ros2_control

18. Configuration Parameters

The main YAML files are:

- config/task_priority_kinematic_control.yaml for the standalone runtime node.
- config/task_priority_controller_ros2_control.yaml for ros2_control.

Important common parameters:

Parameter	Meaning
backend_plugin	Pluginlib name for kinematics backend. Use KDL for runtime.
robot_description	Inline URDF. Usually empty.
robot_description_source_node	Node that owns the robot_description parameter.
world_frame	World frame used in messages and navigation pose interpretation.
base_frame	Base frame used as root for KDL chains.
left_tip_frame, right_tip_frame	Tip frames for default end-effector tasks.
left_arm_joints, right_arm_joints	Joint ordering for the solver.
active_base_dofs	Which of the six base DOFs enter the generalized velocity vector.
solver_method	dls, pinv, or svd.
dls_lambda	DLS damping.
dof_weights	Per-DOF weighted inverse metric.
velocity_limits	Per-DOF command clamp.
task_ids	Ordered names used to load tasks. <id>.* parameters.

19. ROS Messages And Services

Messages

ControllerOutput: detailed command vector publication.

HierarchyState: backend name, solver method, ready flag, and task statuses.

SolverDiagnostics: backend, solver method, DLS lambda, condition estimate, validity, and message.

TaskState: per-task state with target, error, desired velocity, and optional current frame pose.

TaskStatus: service/status summary of configured tasks.

Services

ListTasks: returns task statuses plus backend and solver names.

ReorderTasks: replaces task order and assigns priorities based on the given id list.

SetTaskEnabled: enables or disables a task.

SetTaskDisabled: disables a task by id. This is implemented by the ros2_control controller.

SwitchBackend: switches backend by plugin name. This is implemented by the standalone runtime node.

reset_solver: std_srvs/Trigger service in the standalone runtime node; it calls task_manager_ -> reset().

20. RQT Panel

Package: task_priority_kinematic_control_rqt File:

task_priority_kinematic_control_rqt/task_priority_panel.py

The RQT plugin provides a task-control GUI.

RQT Function-by-Function

`TaskPriorityPanel.__init__(context)`

Initializes RCLPY if needed, creates a node, builds Qt widgets, creates ROS service clients and a hierarchy state subscriber, then refreshes the task list.

`shutdown_plugin()`

Destroys the node and shuts down RCLPY if this plugin initialized it.

`_spin_until_complete(future)`

Spins the plugin node until a service future completes.

`_show_error(message)`

Displays a Qt warning dialog.

`_on_hierarchy_state(msg)`

Updates the top status label with backend, solver, and readiness.

`_refresh_tasks()`

Calls /list_tasks, fills the table, creates enable checkboxes, and records targetable tasks. Pose tasks and joint-array tasks are exposed in the target panel.

`_toggle_task(task_id, state)`

Calls /set_task_enabled and refreshes the table.

`_apply_order()`

Parses comma-separated task ids, calls /reorder_tasks, and refreshes.

`_on_goal_task_changed()`

Switches visible target widgets depending on selected task target type:

- pose tasks show XYZ and quaternion fields
- joint-array tasks show joint names and joint target field
- unsupported tasks disable the send button

`_send_target()`

Publishes either:

- PoseStamped to <TASK_TOPIC_PREFIX>/<task_id>/target
- Float64MultiArray to <TASK_TOPIC_PREFIX>/<task_id>/joint_target

Important namespace note: TASK_TOPIC_PREFIX is hard-coded as

/cirtesub/controller/task_priority/tasks, while the controller-side subscriptions are relative paths such as task_priority/tasks/<id>/target. This can be correct if the controller node namespace resolves to /cirtesub/controller, but it is a key thing to verify during integration.

21. End-to-End Control Tick

1. Navigation arrives:
base_position, base_rpy
2. Joint state arrives:
joint_positions, joint_velocities in model order
3. KDL backend update:
URDF-derived chains + current q -> frame poses/Jacobians
4. Task update:
each task produces J_i , desired_velocity_i, error_i
5. Hierarchy solve:
high priority first, lower priority in null space
6. Command split:
generalized_velocity -> base, left arm, right arm
7. Output:
RuntimeNode publishes topics
or
TaskPriorityController writes ros2_control command interfaces

22. Debugging Checklist

When the controller does not move:

1. Check that robot_description is readable from robot_description_source_node.
2. Check that base_frame, left_tip_frame, and right_tip_frame exactly match URDF frame names.
3. Check that left_arm_joints and right_arm_joints exactly match URDF and ros2_control joint names.
4. Check that navigation has arrived; otherwise base state is invalid.
5. Check that joint states or state interfaces are arriving in finite values.
6. Check /hierarchy_state ready flag.
7. Check task enabled and active flags.
8. Check task priorities after any RQT reorder.
9. Check active_base_dofs; yaw tasks need DOF 5.
10. Check velocity limits; overly small limits can make motion look absent.
11. Check namespace alignment between RQT target publishers and controller target subscribers.

23. Extension Guide

To add a new task:

1. Derive from TaskBaseCommon.
2. Implement configure(...).
3. Implement update(state, backend) returning a valid TaskComputation.
4. Override set_pose_goal or set_joint_target if it accepts runtime goals.
5. Override build_status if the GUI should expose target metadata.
6. Add the source file to CMakeLists.txt.
7. Export the plugin in task_priority_kinematic_control_core_plugins.xml.
8. Add a tasks.<id>. * block to YAML.

To add a real Pinocchio backend:

1. Parse URDF/model in configure.
2. Build a frame index map compatible with configured frame names.

3. In update, compute forward kinematics and frame Jacobians for all needed frames.
4. Return `FrameState` with the same world-frame pose and $6 \times \text{total_dofs}$ Jacobian convention as KDL.
5. Preserve base DOF column ordering exactly, or tasks and solver weights will no longer match.

24. Key Implementation Constraints

- The generalized velocity order is the contract between model, backend, tasks, solver, and output writers.
- KDL computes the real current frame poses and Jacobians; tasks only select rows and compute desired task-space velocities.
- The solver assumes task priority order is already correct.
- The standalone runtime node and `ros2_control` controller duplicate some setup logic. Behavior should be kept aligned when modifying one.
- `EndEffectorPoseTask` and `FramePoseTask` are effectively hold-current tasks until a runtime pose goal arrives.
- `JointLimitsTask` uses its error field as the repulsive command, not as a pure position error.
- The `ros2_control` controller always writes six base command interfaces, even if fewer base DOFs are active in the solver.

25. Mental Model

The package is easiest to understand as three contracts:

Model contract:
names and ordering

Kinematics contract:
`frame_id` -> pose + $6 \times \text{total_dofs}$ Jacobian

Task contract:
task -> desired_velocity + task Jacobian

Solver contract:
ordered task computations -> generalized_velocity

If those contracts stay aligned, the controller can mix base motion, left arm motion, right arm motion, end-effector goals, joint-limit avoidance, and posture bias in one coherent velocity solve.